EXP9-:Implementation of BST using Linked List

Aim

To write and execute a C program to **create a Binary Search Tree using linked list**, perform **Inorder, Preorder and Postorder traversals**, and **delete a specified node** from the Binary Search Tree.

Theory

A **Binary Search Tree (BST)** is a special type of binary tree in which each node contains a data element and two pointers, namely left and right.

Properties of Binary Search Tree

1. All elements in the **left subtree** are **less than** the root node
2. All elements in the **right subtree** are **greater than** the root node
3. The left and right subtrees are also Binary Search Trees
4. No duplicate elements are allowed

Insertion/Creation of BST

Insertion in a BST is done based on comparison:

- If the tree is empty, a new node is created
- If the value is less than the root, it is inserted in the left subtree
- If the value is greater than the root, it is inserted in the right subtree
- The process is repeated recursively until the correct position is found

Tree Traversals

Tree traversal is the process of visiting each node of the tree exactly once.

1. Inorder Traversal (LNR)

- Left subtree → Root → Right subtree
- In BST, inorder traversal displays elements in **ascending order**

2. Preorder Traversal (NLR)

- Root → Left subtree → Right subtree
- Used for copying the tree

3. Postorder Traversal (LRN)

- Left subtree → Right subtree → Root
- Used for deleting the tree

Deletion in Binary Search Tree

Deletion of a node in BST is classified into three cases:

Case 1: Node is a Leaf Node

The node is simply removed from the tree.

Case 2: Node has One Child

The node is deleted and replaced by its child.

Case 3: Node has Two Children

The node is replaced by its **inorder successor** (smallest value in the right subtree), and the successor node is deleted.

Program

```
#include <stdio.h>
#include <stdlib.h>

/* Structure of BST node */
struct node {
    int data;
    struct node *left;
    struct node *right;
};

/* Function declarations */
struct node* insert(struct node *root, int value);
struct node* deleteNode(struct node *root, int key);
struct node* findMin(struct node *root);
void inorder(struct node *root);
void preorder(struct node *root);
void postorder(struct node *root);

int main() {
    struct node *root = NULL;
    int choice, value;

    do {
        printf("\n\n--- BINARY SEARCH TREE MENU ---");
        printf("\n1. Insert");
        printf("\n2. Inorder Traversal");
        printf("\n3. Preorder Traversal");
        printf("\n4. Postorder Traversal");
        printf("\n5. Delete a node");
        printf("\n6. Exit");

        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                root = insert(root, value);
                break;

            case 2:
                if (root == NULL)
                    printf("Tree is empty");
                else {
                    printf("Inorder Traversal: ");
                    inorder(root);
                }
```

```
            break;

        case 3:
            if (root == NULL)
                printf("Tree is empty");
            else {
                printf("Preorder Traversal: ");
                preorder(root);
            }
            break;

        case 4:
            if (root == NULL)
                printf("Tree is empty");
            else {
                printf("Postorder Traversal: ");
                postorder(root);
            }
            break;

        case 5:
            if (root == NULL) {
                printf("Tree is empty");
            } else {
                printf("Enter value to delete: ");
                scanf("%d", &value);
                root = deleteNode(root, value);
            }
            break;

        case 6:
            printf("Exiting program");
            break;

        default:
            printf("Invalid choice");
    }
  } while (choice != 6);

  return 0;
}

/* Insert node into BST */
struct node* insert(struct node *root, int value) {
  if (root == NULL) {
    struct node *newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = value;
    newnode->left = NULL;
    newnode->right = NULL;
    return newnode;
```

```
    }

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

/* Find minimum value node */
struct node* findMin(struct node *root) {
    while (root->left != NULL)
        root = root->left;
    return root;
}

/* Delete node from BST */
struct node* deleteNode(struct node *root, int key) {
    if (root == NULL)
        return root;

    if (key < root->data)
        root->left = deleteNode(root->left, key);

    else if (key > root->data)
        root->right = deleteNode(root->right, key);

    else {
        /* Node with one or no child */
        if (root->left == NULL) {
            struct node *temp = root->right;
            free(root);
            return temp;
        }
        else if (root->right == NULL) {
            struct node *temp = root->left;
            free(root);
            return temp;
        }

        /* Node with two children */
        struct node *temp = findMin(root->right);
        root->data = temp->data;
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

/* Inorder traversal */
```

```
void inorder(struct node *root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

/* Preorder traversal */
void preorder(struct node *root) {
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

/* Postorder traversal */
void postorder(struct node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}
```